



35 steps? great? possible to put all in 1 copy box?

Yes—here's the full 35-step flow in one copy-friendly block.

PHASE 1 – GET THE SOURCE CODE FROM WORDPRESS

▮ Step 1 -- | -- Open All Pages Screen

Action: In wp-admin, click "Pages → All Pages" so you see the list of all pages.^[1]

What Step Does:

- Effect: Shows you every page, including "404 Fail," in one table.^[1]
- Flow: Entry point to open the exact page you're cloning.

▮ Step 2 -- | -- Open 404 Fail Editor

Action: In the Pages list, click the "404 Fail" page title to open its Edit screen.^[1]

What Step Does:

- Effect: Loads the editor for the 404 Fail page.^[2]
- Flow: Moves you into the object you want to clone.

▮ Step 3 -- | -- Confirm Title + URL Match

Action: Check the title field says "404 Fail" and the Permalink shows `https://raverrafting.com/404-fail/`.^[2]

What Step Does:

- Effect: Confirms correct page and live route.^[2]
- Flow: Ensures you don't clone the wrong thing.

▮ Step 4 -- | -- Switch Content To HTML/Code View

Action: Change the editor mode to HTML/Code.

What Step Does:

- Effect: Shows raw HTML markup for the 404 message.
- Flow: Prepares clean copy.

▮ Step 5 -- | -- Copy Full Body Markup

Action: Select all HTML in the content area and copy it (Ctrl+C / Cmd+C).

What Step Does:

- Effect: Puts the complete 404 body code onto your clipboard.^[2]
- Flow: This becomes the main content portion of your cloned view.

PHASE 2 – NOTE LAYOUT + META CONTEXT

▮ Step 6 -- | -- Record Template And Sidebar

Action: In Page Attributes and Page Sidebar, note Template = "Page Sidebar" and Sidebar = "Single Post Sidebar".^[2]

What Step Does:

- Effect: Captures the layout skeleton: main content + right sidebar.^[2]
- Flow: Informs how you structure the layout in your IDE.

▮ Step 7 -- | -- Check Header / Slider / Builder State

Action: In Page Options and Content Builder, check header/slider options and that Content Builder is disabled.^[2]

What Step Does:

- Effect: Confirms this 404 uses normal template rendering, with optional header/slider, no builder grid.^[2]
- Flow: Tells you you only need standard layout in the clone.

▮ Step 8 -- | -- Snapshot SEO / Social / Schema

Action: Look at Yoast snippet preview, Facebook Thumb Fixer object type, Rich Snippet type, and Featured Image state.^[2]

What Step Does:

- Effect: Captures how search engines and social platforms are supposed to see this 404 page.^[2]
- Flow: Gives you a meta blueprint to replicate with head tags/schema in your code.

▮ Step 9 -- | -- Save WordPress State (Optional Safety)

Action: If you made any changes while inspecting, click "Update".^[2]

What Step Does:

- Effect: Ensures the live WordPress 404 Fail page matches the snapshot you captured.^[2]

- Flow: Keeps your clone and the current site consistent for comparison.

PHASE 3 – BUILD THE CLONE IN YOUR IDE

▮ Step 10 -- | -- Create 404 View File In IDE

Action: In your IDE, create/open a dedicated 404 file (e.g., `404.html`, `404.php`, `404.jsx`, `404.tsx`).

What Step Does:

- Effect: Creates the code home for your cloned 404 page.
- Flow: Target file for all code you're about to paste/generate.

▮ Step 11 -- | -- Add Layout Skeleton (Main + Sidebar)

Action: In that file, build a two-column layout: main content on the left and sidebar on the right.

What Step Does:

- Effect: Mirrors the Page Sidebar + Single Post Sidebar structure.^[2]
- Flow: Gives matching structure before you add content.

▮ Step 12 -- | -- Paste WordPress HTML Into Main Area

Action: Paste the copied 404 HTML into the main content area (or a `NotFoundContent` component).

What Step Does:

- Effect: Recreates the exact 404 message and image inside your new layout.
- Flow: Ensures your cloned 404 visually matches the original content.

▮ Step 13 -- | -- Define Sidebar Placeholder Content

Action: In the sidebar region, add placeholder widgets (latest posts, categories, events, ads) matching the spirit of "Single Post Sidebar".^[2]

What Step Does:

- Effect: Establishes a right-rail zone similar to the WordPress sidebar.
- Flow: Ready for real data wiring later.

PHASE 4 – HOOK UP ROUTING AND META

▮ Step 14 -- | -- Connect 404 Route To This View

Action: In your router/server config, add a rule that sends all unknown paths (404s) to this 404 file/component.

What Step Does:

- Effect: Makes this cloned view respond when a route is missing, like WordPress 404 behavior.
- Flow: Connects broken URL → your new 404 page.

▮ **Step 15 -- | -- Set Page Title + Description In Head**

Action: Use your framework's head/metadata API to set a title (e.g., "404 Fail | RaverRafting") and meta description similar to the Yoast snippet.^[2]

What Step Does:

- Effect: Rebuilds SEO intent for the 404 page in your environment.
- Flow: Keeps search/link previews aligned with the WordPress original.

▮ **Step 16 -- | -- Add Open Graph Tags (OG)**

Action: Add OG tags (og:title, og:description, og:type, og:image) for the 404 page.

What Step Does:

- Effect: Replaces Facebook Thumb Fixer + featured image behavior with explicit OG config.^[2]
- Flow: Ensures social shares of the 404 route look intentional.

▮ **Step 17 -- | -- Optional: Add Schema / JSON-LD**

Action: Add a JSON-LD block declaring a `WebPage` or `Article` representing this 404 page.

What Step Does:

- Effect: Takes over the Rich Snippet role for structured data.^[2]
- Flow: Future-proofs the 404 for rich-result/analytics usage.

PHASE 5 – VERIFY THE CLONE (ACCURACY CHECK)

▮ **Step 18 -- | -- Visual Side-By-Side Check**

Action: Open <https://raverrafting.com/404-fail/> and your new 404 route; visually compare layout, content, key visuals.

What Step Does:

- Effect: Confirms the cloned page looks and reads like the WordPress original.
- Flow: Catches obvious mismatches (missing image, wrong text, layout differences).

▮ **Step 19 -- | -- Behavior Check For Bad URLs**

Action: Hit a clearly wrong URL in your new app (e.g., `/this-page-does-not-exist`) and confirm it routes to your 404 view.

What Step Does:

- Effect: Verifies the 404 routing rule works.
- Flow: Ensures broken URLs land on your cloned 404 instead of a default server error.

▮ Step 20 -- | -- Meta / Social Check

Action: Use dev tools or a sharing debugger to inspect title, description, OG tags, and schema on your new 404 route.

What Step Does:

- Effect: Confirms meta + structured data are set correctly.
- Flow: Completes the clone from user view to crawler/social view.

PHASE 6 – TURN THIS INTO A REUSABLE PATTERN

▮ Step 21 -- | -- Abstract A Generic Layout Component

Action: Refactor your 404 layout into a reusable `SitePageLayout` that accepts props like `hasSidebar`, `showHeader`, `pageTitle`.

What Step Does:

- Effect: Turns the 404 layout shell into a generic building block.
- Flow: Makes future page clones (About, Contact, etc.) faster.

▮ Step 22 -- | -- Abstract A Generic SEO Helper

Action: Generalize your 404 meta helper into `setPageMeta({ title, description, ogImage, type })`.

What Step Does:

- Effect: Converts 404-specific meta logic into a reusable meta engine.
- Flow: Lets any cloned page share one meta pattern.

▮ Step 23 -- | -- Encapsulate 404 As Data + Template

Action: Separate the 404 data (title, description, hero image URL, body HTML) from the component into a config consumed by the 404 page.

What Step Does:

- Effect: Treats 404 as “content config + generic template” instead of a unique snowflake.
- Flow: Sets the model for feeding other pages into the same template.

▮ Step 24 -- | -- Define A “PageCloneSpec” Schema

Action: Create an interface/JSON schema for `PageCloneSpec` with fields: `slug`, `title`, `layoutType`, `hasSidebar`, `meta`, `bodyHtml`.

What Step Does:

- Effect: Formalizes the data you need from each WordPress page.
- Flow: Becomes your standard checklist for future clones.

▮ **Step 25 -- | -- Document The 404 Clone Spec**

Action: Fill out a `PageCloneSpec` object for the 404 Fail page using the info you already captured.

What Step Does:

- Effect: Creates a concrete config record describing the 404 in the new system.
- Flow: Serves as a live example for your AI coder and future you.

PHASE 7 – PREP FOR OTHER PAGES (REPEATABILITY)

▮ **Step 26 -- | -- Identify Next Target Pages**

Action: On the Pages list, pick 3–5 other key pages (Home, About, Contact, important guides) to clone next.^[1]

What Step Does:

- Effect: Defines a small batch of priority pages for migration.
- Flow: You'll run this same pattern on each.

▮ **Step 27 -- | -- Create A "Clone Checklist" Template**

Action: Turn Steps 1–20 into a generic text checklist that says "Target Page" instead of "404 Fail."

What Step Does:

- Effect: Converts your 404 process into a reusable recipe.
- Flow: Reduces cognitive load for the next clones.

▮ **Step 28 -- | -- Set Up A `pages-spec.json` File**

Action: In your repo, create `pages-spec.json` (or TS config) and add the 404 spec as the first entry.

What Step Does:

- Effect: Provides a central place where all `PageCloneSpecs` live.
- Flow: Enables automation (scripts that generate components from specs).

▮ **Step 29 -- | -- Create A "Scaffold From Spec" Script**

Action: Write or have AI generate a Node/TS script that reads `pages-spec.json` and outputs skeleton components/layout wiring.

What Step Does:

- Effect: Begins automating boilerplate component creation from specs.
- Flow: Makes the 2nd, 3rd, 10th page clone much cheaper.

▮ **Step 30 -- | -- Dry Run Script On 404 Spec**

Action: Run the scaffold script using just the 404 spec and compare the generated component to your handcrafted 404.

What Step Does:

- Effect: Validates that your automation matches the manual version.
- Flow: Builds trust in the script for future pages.

PHASE 8 – CONSISTENCY + QA ACROSS ENVIRONMENTS

▮ **Step 31 -- | -- Add A Visual Regression Checklist**

Action: Create a 7-item checklist for every cloned page: title, H1, body copy, key images, layout (columns), spacing, links.

What Step Does:

- Effect: Standardizes how you verify parity.
- Flow: Prevents you from missing important pieces across pages.

▮ **Step 32 -- | -- Connect Clone Specs To Routes Map**

Action: Add a `routes.ts` (or similar) that maps each `PageCloneSpec.slug` to a route in your app.

What Step Does:

- Effect: Centralizes routing decisions linked to specs.
- Flow: Avoids scattered route definitions as you grow.

▮ **Step 33 -- | -- Log A “Cloning Status” Table**

Action: Make a Markdown table with columns: Page, WP URL, Spec Created, Component Generated, Route Wired, QA Passed.

What Step Does:

- Effect: Tracks migration progress visually.
- Flow: Gives you a mini dashboard as you Anti-Gravity the whole site.

▮ **Step 34 -- | -- Document Intended Differences**

Action: For each cloned page, jot a short “Differences vs original” note (intentional design/content changes).

What Step Does:

- Effect: Documents what's deliberately different vs what might be a bug.
- Flow: Future-you and collaborators know what not to "fix back."

▯ **Step 35 -- | -- Plan A Performance / UX Second Pass**

Action: Make a small list of optimizations to run after cloning (image optimization, lazy loading, CSS cleanup, accessibility checks) and schedule a pass.

What Step Does:

- Effect: Moves you from "it's cloned" to "it's sharper and faster than the original."
- Flow: Completes the Anti Gravity goal: not just copying RaverRafting, but improving it.

✱✱

1. https://raverrafting.com/wp-admin/edit.php?post_type=page
2. <https://raverrafting.com/wp-admin/post.php?post=14780&action=edit>